

ARM PrimeCell Driver

SMC v1.0 (PL092)

Test Report

Primecell Driver Test Report

© Copyright ARM Limited 2001. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, SMCeI Gen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Document Summary

Document Issue	A
Primecell driver identifier	SMC
Code Version	1.0
Primecell identifier	PL092
Test date	30, August, 2001

Contents

1	ABOUT THIS DOCUMENT	5
1.1	Change history	5
2	RELEASED FILES	6
3	TEST ENVIRONMENT	6
3.1	Test Specification Summary	6
3.2	Hardware requirements	7
4	CODE COVERAGE	8
4.1	Overall	8
4.2	Untested Blocks	8
4.3	Untested Routine Exits	9
5	RELEASED TEST MODULE	10
5.1	Overview	10
5.2	Changes to test environment	10
5.3	Configuration of the test module	10
5.4	Outline	10
5.5	Results	12
5.6	Performance obtained	12

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A	30, August, 2001	First release of test report

2 RELEASED FILES

PL092-ISPC-1000-A	API document
PL092-TREP-1000	Test Results document
apSMC/apSMC.h	Public header file
apSMC/SMC.h	Private header file
apSMC/SMC.c	Source code
apSMC/SMCtst.c	Released test code

3 TEST ENVIRONMENT

3.1 Test Specification Summary

	Specification (Intended)	Actual Test Environment If different from Specification
Code Test Platform	INTEGRATOR	
Core used for test	ARM7TDMI	
Module ROM size (RO code + RO data)	840 Bytes	
Module RAM size (ZI data) per instance	8 Bytes	
PrimeCell tested	PL092	
External hardware used	N/A	
Performance obtained	N/A	
Driver modules	SMC.c	
Test modules	SMCtst.c	
Standard modules required (normal archive location)	os.c, oss.s, int.c, dispatch.s, boot.s, vectors.s, test.c, main.c	
Custom modules required (in [project]/privates/apSMC)	applatfm.h	

3.2 Hardware requirements

3.2.1 FPGA

Is the image used to program the FPGA archived	NO
If so, at what location	N/A

3.2.2 Test configuration

Is the hardware configuration documented	NO
If so, at what location	N/A

FPGA setup:

SMC base address:	0xFF000000
-------------------	------------

3.2.3 External connectors required

none

3.2.4 Devices on prototyping area

none

3.2.5 External test equipment

none

4 CODE COVERAGE

4.1 Overall

Code coverage tests have been performed (generated by v1.18) YES

Code coverage details below

- ◆ *Clear the data area below the application base address.*
- ◆ *Run all test applications.*

Percentage of normal routine exits tested. 44%

- ◆ *Examine all routine exits listed as untested to ensure that these are not used in normal flow.*

All untested routine exits examined. YES

Percentage of blocks tested. 47%

Code Coverage tests output:

Code Coverage report

=====

Routine returns:

Missed: CC(R,3) CC(R,4) CC(R,5) CC(R,6) CC(R,7) CC(R,8) CC(R,10) CC(R,12) CC(R,14) CC(R,16)

Code coverage: 44 %

Blocks:

Missed: CC(B,5) CC(B,6) CC(B,7) CC(B,8) CC(B,9) CC(B,10) CC(B,12) CC(B,14) CC(B,16) CC(B,18)

Code coverage: 47 %

4.2 Untested Blocks

Blocks not covered by these tests have been examined and categorised as below.

Block number	Routine	Reason
5	apSMC_MemBankConfigSet	This only happens when an invalid memory bank number is being sought
6	apSMC_MemBankConfigSet	This only happens when an invalid value of wait states for read access is being sought
7	apSMC_MemBankConfigSet	This only happens when an invalid value of wait states for write

		access is being sought
8	apSMC_MemBankConfigSet	This only happens when an invalid value of duty or turn-around cycles is being sought
9	apSMC_MemBankConfigSet	This only happens when an invalid value of output enable assertion delay is being sought
10	apSMC_MemBankConfigSet	This only happens when an invalid value of write enable assertion delay is being sought
12	apSMC_MemBankConfigGet	This only happens when an invalid memory bank number is being sought
14	apSMC_ErrorGet	This only happens when an invalid memory bank number is being sought
16	apSMC_WriteEnable	This only happens when an invalid memory bank number is being sought
18	apSMC_WriteDisable	This only happens when an invalid memory bank number is being sought

4.3 Untested Routine Exits

Untested routine exits not covered by these tests have been examined and categorised as below.

Exit number	Routine	Reason
3	apSMC_MemBankConfigSet	This only happens when an invalid memory bank number is being sought
4	apSMC_MemBankConfigSet	This only happens when an invalid value of wait states for read access is being sought
5	apSMC_MemBankConfigSet	This only happens when an invalid value of wait states for write access is being sought
6	apSMC_MemBankConfigSet	This only happens when an invalid value of duty or turn-around cycles is being sought is being sought
7	apSMC_MemBankConfigSet	This only happens when an invalid value of value of output enable assertion delay is being sought
8	apSMC_MemBankConfigSet	This only happens when an invalid value of write enable assertion delay is being sought
10	apSMC_MemBankConfigGet	This only happens when an invalid memory bank number is being sought
12	apSMC_ErrorGet	This only happens when an invalid memory bank number is being sought
14	apSMC_WriteEnable	This only happens when an invalid memory bank number is being sought
16	apSMC_WriteDisable	This only happens when an invalid memory bank number is being sought

5 RELEASED TEST MODULE

5.1 Overview

The released test module is named `SMCtst.c`.

It is used as a self-test module for the SMC PrimeCell and also as a part of the formal testing.

Test environment is as described earlier YES

Code coverage: Percentage of blocks tested by this test module 47%

5.2 Changes to test environment

None for public tests.

5.3 Configuration of the test module

The test module is configured through the macro `apSMC_BANK7_BOOT` in system description file `applatfm.h`.

If `apSMC_BANK7_BOOT` is defined in `applatfm.h`, that indicates that system boots from Bank 7, this memory bank is being excluded from the test, and number of banks for test is set up to 7. Otherwise, if `apSMC_BANK7_BOOT` is not defined, Bank 7 is being tested, and number of memory banks is set up to 8.

Macro `apINT_VERSION` in `apintcfg.h` should be defined as follows:

```
# define apINT_VERSION    apVERSION_NONE_ON_LM
```

5.4 Outline

The test module performs the following functions:

5.4.1 Initialisation

- ⊕ Call to `apSMC_Initialize()` function to set up peripheral base address.
 - Number of interrupts used 0
 - Specific parameters passed in configuration data NONE
- ⊕ Call to `apSMC_StateSizeGet()` function to get size required for storage of the driver state data.

This function is called for test only and return value is not used.

5.4.2 Test Operation

This section is the main body of the test and performs test operations of the driver. After SMC initialisation it configures all memory banks with different parameters, reads values of bank registers and then compares them with programmed parameters.

- ⊕ Call to `apSMC_MemBankConfigSet()` function to configure memory bank.
- ⊕ Call `apSMC_MemBankConfigGet()` function to read configuration parameters of memory bank.

-
- ⊕ Call `apSMC_InitGet()` function to get initialisation flag of the memory bank.
 - ⊕ Call `apSMC_WriteEnable()` function to enable write operations for the bank.
 - ⊕ Call `apSMC_MemBankConfigGet()` function to read write protect parameter of the memory bank.
 - ⊕ Call `apSMC_WriteDisable()` function to disable write operations for the bank.
 - ⊕ Call `apSMC_MemBankConfigGet()` function to read write protect parameter of the memory bank.
 - ⊕ Call `apSMC_ErrorGet()` function to read error status flags of the memory bank.

5.4.3 Results

The possible results for this test are as follows:

If the test detects that values read from bank registers do not match to configuration parameters, then the following message will be displayed:

```
Testing Memory Bank: N Read Error Flags: None : Fail
```

, where

N – the number of the memory bank.

If the test detects that read error status flags are set, then the following message will be displayed:

```
Testing Memory Bank: N Read Error Flags: WaitToutErr WriteProtErr HSizeErr: Pass
```

, where

N – the number of the memory bank.

The following list the possible results for this test:

Testing module SMC at 0xff000000

Testing Memory Bank: 0 Read Error Flags: None : Pass

Testing Memory Bank: 1 Read Error Flags: None : Pass

Testing Memory Bank: 2 Read Error Flags: None : Pass

Testing Memory Bank: 3 Read Error Flags: None : Pass

Testing Memory Bank: 4 Read Error Flags: None : Pass

Testing Memory Bank: 5 Read Error Flags: None : Pass

Testing Memory Bank: 6 Read Error Flags: None : Pass

Testing Memory Bank: 7 Read Error Flags: None : Pass

*** PASS ***

5.5 Results

Test run returns TRUE	YES
When apTEST_INTERACTIVE is FALSE, no user interaction is required	YES
When apTEST_VERBOSE is FALSE, no output is produced	YES

5.6 Performance obtained

Performance obtained in this test	N/A
-----------------------------------	-----

